

77. Pourquoi une fonction ne s'appelle pas elle-même

Le refus le plus important du projet — et comment dérouler la boucle.

1. Ce qu'il faut comprendre

Écrire `return n + somme(n - 1);` est **refusé**, avec le cycle nommé : « `somme` finit par s'appeler elle-même (`somme` → `somme`) ».

Les arguments vivent à une adresse fixe, réservée une fois pour toutes — c'est ce que fait tout compilateur C d'une machine huit bits, faute d'une pile praticable. Un appel imbriqué écraserait les arguments de l'appel en cours, et la fonction reprendrait son travail avec les valeurs de l'autre.

C'est un bogue silencieux, et indébogable. Le compilateur le nomme avant qu'il n'arrive. Voici la même chose, déroulée avec `while`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **580 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
somme()         31 o ← nombre() - ta fonction
main()          33 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte     10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          256 o ← CopierTuiles - 16 lettres sur 43
"SOMME DE 1 A 5" 14 o ← texte() - un texte gravé
```

Lettres gravées : ADEMOS0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 1    8 o sur place, dans main() + somme() (31 o), EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

```
Tes fonctions appelées :  
somme() x 1 7 0 sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : calcule 1 + 2 + 3 + 4 + 5 avec une boucle, et affiche 15.  
//  
// Ici, une fonction n'a PAS le droit de s'appeler elle-même : le compilateur  
// le refuse (ses arguments sont rangés à une place fixe ; un deuxième appel  
// les écraserait). On fait donc le calcul avec une boucle while.  
  
#include <texte> // écrit un texte à l'écran  
#include <nombre> // écrit un nombre en chiffres
```

2. UNE FONCTION — SOMME()

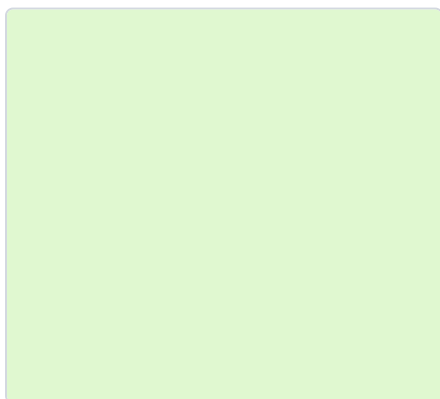
```
uint8_t somme(uint8_t n) { // Rend 1 + 2 + ... + n.  
    uint8_t total = 0; // Le total commence à 0.  
    while (n > 0) { total += n; n--; }  
    // Tant que n > 0 : on ajoute n au total, puis n perd 1.  
    // n = 5 : total 5 ; n = 4 : 9 ; n = 3 : 12 ;  
    // n = 2 : 14 ; n = 1 : 15 ; n = 0 : on sort.  
    return total; // La réponse : 15.  
}
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

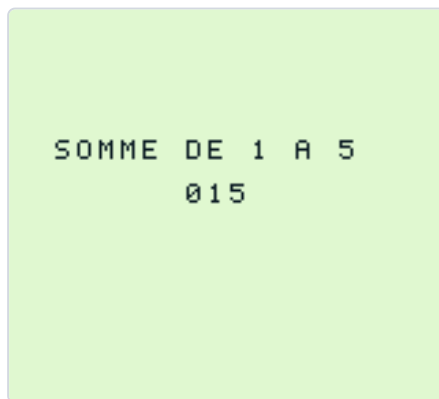
```
int main() { // Le jeu commence ici.  
    texte(2, 6, "SOMME DE 1 A 5");  
    nombre(8, 8, somme(5)); // Affiche 015.  
  
    while (true) { // Répète pour toujours :  
        image(); // attend l'image suivante.  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la somme est juste

✓ l'explication est écrite

Ensuite : **78. #include, et les onglets de la page** — chapitre 11 : Découper son programme. · [gameboy3](#) — écrire une cartouche Game Boy en C++